

Convolutional Neural Networks

September 8, 2026

Overview

In a typical Convolutional Neural Network (CNN) for image classification, the input image is processed through several layers to extract features and produce a prediction. For a 10-class classification task, the general steps are:

1. **Input and Convolution:** The network takes an input image of size $C \times H \times W$ (where C is the number of channels, and H and W are the height and width). A convolutional layer applies $\bar{F}$ filters of size $f \times f$ with a given stride s and padding p . This produces feature maps.
2. **Downsampling:** Pooling (e.g., 2×2 max pooling) or strided convolutions reduce the spatial dimensions, thereby increasing the receptive field and reducing computational cost.
3. **Final Layers:** The final feature maps are aggregated (by flattening or global pooling), and then one or more fully connected (dense) layers produce a 10-dimensional output, which is passed through a softmax function to obtain class probabilities.
4. **Loss:** Cross-entropy loss is typically used to train the network, comparing the predicted class probabilities with the true labels.

Convolution Layer

Input

Assume an input image with shape:

$$(C, H, W)$$

For example, an RGB image has $C = 3$.

Filters (Kernels)

We apply $\bar{F}$ filters (kernels) of size $f \times f$. Each filter spans all channels, so each filter is of shape:

$$(C, f, f)$$

and contains $C \times f \times f$ weights (plus a bias term per filter).

Padding and Stride

- **Padding p :** Adding zeros around the border of the image. This can be used to preserve spatial dimensions (“same” padding) or to control output size.
- **Stride s :** The step size of the filter when it moves across the input. For example, stride $s = 1$ moves the filter one pixel at a time; $s > 1$ produces a smaller output.

Output Size of Convolution

The spatial dimensions of the output feature map are given by:

$$H_{\text{out}} = \frac{H - f + 2p}{s} + 1, \quad W_{\text{out}} = \frac{W - f + 2p}{s} + 1.$$

Since we apply $\bar{F}$ filters, the output of the convolutional layer has shape:

$$(\bar{F}, H_{\text{out}}, W_{\text{out}}).$$

Example: For an input of size $(3, 32, 32)$, $\bar{F} = 7$, $f = 3$, $s = 1$ and no padding ($p = 0$), the spatial dimensions become:

$$H_{\text{out}} = 32 - 3 + 1 = 30, \quad W_{\text{out}} = 30.$$

Thus, the output shape is:

$$(7, 30, 30).$$

Downsampling Operations

Downsampling reduces the spatial dimensions of the feature maps, which is useful for:

- **Reducing computational complexity** and memory usage.
- **Increasing the receptive field** of later layers.
- **Introducing spatial invariance** to small translations and distortions.
- **Controlling overfitting** by reducing the number of activations.

Pooling Layers

Max Pooling (e.g., 2×2 with stride 2):

- **Operation:** The 2×2 window slides over the input feature map (typically without overlap). In each window, the maximum value is chosen.

- **Output Size:** If the input to pooling is $H_{\text{in}} \times W_{\text{in}}$, the output becomes:

$$H_{\text{pool}} = \left\lfloor \frac{H_{\text{in}}}{2} \right\rfloor, \quad W_{\text{pool}} = \left\lfloor \frac{W_{\text{in}}}{2} \right\rfloor.$$

Strided Convolutions

An alternative to pooling is to use a convolution with a stride greater than 1. For example, a convolution with stride $s = 2$ reduces the spatial dimensions by roughly half while simultaneously learning a feature transformation.

How Far to Downsample?

- The goal is to gradually reduce the spatial dimensions while retaining essential features.
- Excessive downsampling may lose critical spatial details; too little downsampling can lead to high computational costs and overfitting.
- A common strategy is to reduce the dimensions by a factor of 2 several times (e.g., $32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 1$), then flatten or apply global pooling before fully connected layers.

Final Layers and Loss Function

Feature Aggregation

After several convolution and downsampling layers, the spatial feature maps are aggregated:

- **Flattening:** Reshape the feature maps into a single vector.
- **Global Pooling:** Compute a statistic (average or max) over the entire spatial dimensions to yield one value per channel.

Fully Connected Layers

The aggregated vector is fed into one or more dense layers that transform the features to a vector of size 10 (for 10 classes).

Output and Loss

- The final layer produces logits, which are converted into class probabilities using a softmax function.
- For classification, the **cross-entropy loss** is typically used. If $\hat{y}$ is the predicted probability vector and y is the one-hot encoded true label, the loss is:

$$L = - \sum_{i=1}^{10} y_i \log(\hat{y}_i).$$

Putting It All Together: An Example Pipeline

1. **Input:** An image of shape (C, H, W) .
2. **Convolution:** Apply $\bar{F}$ filters of size $C \times f \times f$ with stride s and padding p . Output shape: $(\bar{F}, \frac{H-f+2p}{s} + 1, \frac{W-f+2p}{s} + 1)$. **Example:** For $s = 1, p = 0$ and $f = 3$, output shape is $(\bar{F}, H - 2, W - 2)$.
3. **Downsampling:** Apply 2×2 max pooling with stride 2. New shape: $(\bar{F}, \lfloor \frac{H-2}{2} \rfloor, \lfloor \frac{W-2}{2} \rfloor)$.
4. **Additional Layers:** Repeat convolutional and downsampling layers as needed to build a hierarchy of features.
5. **Aggregation:** Use flattening or global pooling to convert the final feature maps into a fixed-length vector.
6. **Classification:** Pass the vector through fully connected layers to obtain a 10-dimensional output. Apply softmax to get class probabilities.
7. **Loss:** Use cross-entropy loss to compare predictions with true labels.

Once the convolution operation has been applied, the standard sequence of processing is:

1. Convolution: Apply N filters to generate N feature maps.
2. Activation Function (e.g., ReLU): Apply non-linearity to introduce feature transformations.
3. Downsampling (if applicable) (e.g., pooling): Reduce spatial dimensions to retain important features and reduce computation.

Design Considerations

- **Downsampling Trade-off:** Too little downsampling results in high computational cost and a high-dimensional feature vector; too much may lose important spatial details.
- **Gradual Reduction:** It is common to reduce spatial dimensions gradually (e.g., halving them repeatedly) to maintain a balance between information retention and efficiency.
- **End-to-End Training:** The entire network (convolutional layers, pooling layers, fully connected layers) is trained end-to-end via backpropagation.

Additional Note: Tokenization in Vision vs. Text

In text processing, we start by tokenizing words (or subwords) and then mapping these tokens to dense embeddings using an embedding lookup (starting from one-hot representations). In vision

tasks, an image is tokenized by splitting it into patches. Each patch is inherently a continuous-valued vector (e.g., its RGB pixel values) and is directly projected into a dense embedding via a linear layer. Though the mechanisms differ, both approaches aim to convert raw input into a set of tokens that are represented as dense vectors for further processing.

CNNs for Text Processing

Although CNNs are primarily used for image processing, they are also effective in text-based tasks, such as sentiment analysis, text classification, and machine translation. Instead of processing spatial features, CNNs for text analyze sequential patterns and n-gram features.

Consider a sentence as a sequence of words represented as embeddings:

$$X = \{x_1, x_2, \dots, x_T\}, \quad \text{where } x_t \in \mathbb{R}^d.$$

where T is the sequence length and d is the embedding dimension.

Convolution Over Text

A 1D convolution operation applies a filter $W \in \mathbb{R}^{f \times d}$ over f consecutive word embeddings. The convolution at position t is:

$$c_t = f(W \cdot X_{t:t+f-1} + b),$$

where:

- $X_{t:t+f-1}$ is a segment of f consecutive embeddings,
- W is the filter matrix,
- b is the bias term,
- $f(\cdot)$ is the activation function (e.g., ReLU).

Pooling Operation

After convolution, pooling is applied to reduce the dimensionality. The most common approach is max-pooling, which selects the most significant feature:

$$\hat{c} = \max(C), \quad C = \{c_1, c_2, \dots, c_{T-f+1}\}.$$

This results in a fixed-size feature vector that can be fed into a fully connected layer for classification.

In detail, we have the following steps:

Step 1: Input Representation - Word Embeddings

We start with a **sentence of length** $T = 10$ (10 words), where each word is represented as a **300-dimensional embedding**.

$$X \in \mathbb{R}^{T \times d} = \mathbb{R}^{10 \times 300}$$

Each row $x_i \in \mathbb{R}^{300}$ represents a **word embedding**.

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{10} \end{bmatrix}, \quad x_i \in \mathbb{R}^{300}$$

Step 2: Convolution Layer - Extracting Local Features

We apply **3 different convolution filters** that detect **n-grams**:

1. **Bigram filter** $\rightarrow$ Captures **2-word** interactions ($k = 2$)
2. **Trigram filter** $\rightarrow$ Captures **3-word** interactions ($k = 3$)
3. **4-gram filter** $\rightarrow$ Captures **4-word** interactions ($k = 4$)

Each filter has a shape:

$$W_k \in \mathbb{R}^{k \times d} = \mathbb{R}^{k \times 300}$$

The **bias term** for each filter is a single scalar:

$$b_k \in \mathbb{R}$$

Convolution Computation

Each filter slides over the sentence, extracting **n-gram representations**.

For **bigram filtering** ($k = 2$), at position i :

$$X_{i:i+2-1} \in \mathbb{R}^{2 \times 300}$$

Filter operation:

$$z_i^{(1)} = f(W_1 \cdot X_{i:i+2-1} + b_1)$$

where:

- $W_1 \in \mathbb{R}^{2 \times 300}$
- $X_{i:i+1} \in \mathbb{R}^{2 \times 300}$
- $b_1 \in \mathbb{R}$
- $z_i^{(1)} \in \mathbb{R}$ (a single scalar output)

Similarly, for **trigram filtering** ($k = 3$), at position i :

$$X_{i:i+3-1} \in \mathbb{R}^{3 \times 300}$$

Filter operation:

$$z_i^{(2)} = f(W_2 \cdot X_{i:i+3-1} + b_2)$$

where:

- $W_2 \in \mathbb{R}^{3 \times 300}$
- $X_{i:i+2} \in \mathbb{R}^{3 \times 300}$
- $b_2 \in \mathbb{R}$
- $z_i^{(2)} \in \mathbb{R}$

For **4-gram filtering** ($k = 4$), at position i :

$$X_{i:i+4-1} \in \mathbb{R}^{4 \times 300}$$

Filter operation:

$$z_i^{(3)} = f(W_3 \cdot X_{i:i+4-1} + b_3)$$

where:

- $W_3 \in \mathbb{R}^{4 \times 300}$
- $X_{i:i+3} \in \mathbb{R}^{4 \times 300}$
- $b_3 \in \mathbb{R}$
- $z_i^{(3)} \in \mathbb{R}$

What Kind of Product is Done Between W_k and $X_{i:i+k-1}$ in CNNs for Text?

When we apply a **convolutional filter** W_k to a submatrix of the input embeddings $X_{i:i+k-1}$, we are performing a **matrix-vector dot product** between the filter and the selected word embeddings.

1. Dimensions of W_k and $X_{i:i+k-1}$

For a **k-gram filter**, we define:

- $W_k \in \mathbb{R}^{k \times d} \rightarrow$ A weight matrix for an n-gram filter.
- $X_{i:i+k-1} \in \mathbb{R}^{k \times d} \rightarrow$ A window of consecutive word embeddings.

For example, with:

- $k = 3$ (trigram filter)
- $d = 300$ (word embedding dimension)

We get:

- $W_3 \in \mathbb{R}^{3 \times 300}$
- $X_{i:i+3-1} \in \mathbb{R}^{3 \times 300}$

2. Operation: Element-wise Multiplication and Summation (Dot Product)

The convolution operation applies a **dot product** between W_k and $X_{i:i+k-1}$:

$$z_i = \sum_{j=1}^k W_k[j, :] \cdot X_{i+j-1, :} + b$$

where:

- $W_k[j, :] \rightarrow$ The **j-th row** of the filter matrix.
- $X_{i+j-1, :} \rightarrow$ The **embedding vector** of the corresponding word.
- $b \in \mathbb{R} \rightarrow$ A scalar bias term.

Each row in W_k corresponds to **one word position** in the filter window, and each row in $X_{i:i+k-1}$ corresponds to **one word embedding**.

Thus, at each position i , the result z_i is a **scalar**.

3. Explicit Example: Computing z_i by Hand

Suppose we have:

$$X_{i:i+2} = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & x_1^{(3)} \\ x_2^{(1)} & x_2^{(2)} & x_2^{(3)} \\ x_3^{(1)} & x_3^{(2)} & x_3^{(3)} \end{bmatrix}$$

where each $x_j^{(d)}$ is a feature from the embedding.

Let's say:

$$W_3 = \begin{bmatrix} w_1^{(1)} & w_1^{(2)} & w_1^{(3)} \\ w_2^{(1)} & w_2^{(2)} & w_2^{(3)} \\ w_3^{(1)} & w_3^{(2)} & w_3^{(3)} \end{bmatrix}$$

Then, the convolution computes:

$$z_i = \sum_{j=1}^3 \sum_{d=1}^3 w_j^{(d)} x_j^{(d)} + b$$

This is essentially:

- A **dot product per row**, then summing across rows.
- Equivalent to **flattening** the matrices and taking a **vector dot product**.

Step 3: Feature Map Dimensions After Convolution

Each filter outputs a **feature map** by sliding over the sentence.

Filter Type	Filter Shape (W_k)	Bias (b_k)	Feature Map Shape
Bigram ($k = 2$)	2×300	$\mathbb{R}$	$\mathbb{R}^{9 \times N}$
Trigram ($k = 3$)	3×300	$\mathbb{R}$	$\mathbb{R}^{8 \times N}$
4-gram ($k = 4$)	4×300	$\mathbb{R}$	$\mathbb{R}^{7 \times N}$

where N is the number of filters per size.

Step 4: ReLU Activation

ReLU is applied **element-wise** to introduce non-linearity:

$$h_i = \max(0, z_i)$$

The feature map **shape remains the same**.

Step 5: Max-Pooling Layer

We apply **1-max pooling** over each feature map.

Since our feature maps have varying lengths:

- **Bigram max-pool** $\rightarrow$ selects **1 value** from $9 \times N \rightarrow \mathbb{R}^N$
- **Trigram max-pool** $\rightarrow$ selects **1 value** from $8 \times N \rightarrow \mathbb{R}^N$
- **4-gram max-pool** $\rightarrow$ selects **1 value** from $7 \times N \rightarrow \mathbb{R}^N$

Thus, the final **concatenated feature vector**:

$$p = \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} \in \mathbb{R}^{3N}$$

In the context of **CNNs for text processing**, N refers to the **number of filters (kernels) used per n-gram size** in the convolutional layer.

Where Does N Appear?

1. Each filter detects **specific patterns** (e.g., bigrams, trigrams).
2. If we use N filters per n-gram size, we get N feature maps per filter size.

For example, if:

- We use **bigram (2-gram)**, **trigram (3-gram)**, and **4-gram** filters.
- Each has $N = 100$ filters.

Then, the total **number of extracted features after max-pooling** is:

$$\underbrace{100}_{\text{bigrams}} + \underbrace{100}_{\text{trigrams}} + \underbrace{100}_{\text{4-grams}} = 300.$$

This **final feature vector of size $3N$** is then passed to the fully connected layer for classification.

Step 6: Fully Connected Layer

We pass the feature vector into a **fully connected dense layer**:

$$\hat{y} = W_{\text{fc}}p + b_{\text{fc}}$$

where:

- $W_{\text{fc}} \in \mathbb{R}^{3N \times C}$ (C = number of classes, e.g., **positive/negative**).
- $b_{\text{fc}} \in \mathbb{R}^C$.

Output **before softmax**:

$$\hat{y} \in \mathbb{R}^C$$

Step 7: Softmax Output

We normalize the outputs into probabilities:

$$P(y = c|X) = \frac{e^{\hat{y}_c}}{\sum_j e^{\hat{y}_j}}$$

Final probability distribution: $\mathbb{R}^C$.

Step 8: Loss Function (Cross-Entropy)

For **binary classification**:

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

where:

- y_i is the true label (one-hot encoded).
- $\hat{y}_i$ is the predicted probability.

Final Summary: Dimensions at Each Step

Layer	Input Shape	Output Shape
Input Embeddings	10×300	10×300
Convolution (N filters)	10×300	$(T - k + 1) \times N$
ReLU Activation	$(T - k + 1) \times N$	$(T - k + 1) \times N$
Max-Pooling	$(T - k + 1) \times N$	$1 \times N$
Concatenation	$3 \times N$	$3N$
Fully Connected	$3N$	C
Softmax Output	C	C